

Inverted Index Construction

Ci si chiede **come costruire un inverted index sapendo di avere a disposizione spazio limitato in RAM?**

Anzitutto si ricorda come avviene la costruzione dell'inverted index: A partire dai documenti base, si fa preprocessing (lemmatizzazione, stopword removal...) e si estraggono quindi i termini con i rispettivi document ID. Dopodiché il passo fondamentale è **ordinare** per termine l'intero insieme di coppie (termine, document ID).

Usando ad esempio Quicksort (che riordina in-place con tempo medio $O(n \log n)$) si dovrebbe mantenere in memoria un array di dimensione $O(n)$, ma se il numero di termini è molto elevato, potrebbe non entrare in RAM.

Vedremo come esempio su cui applicare algoritmi scalabili in RAM la Reuters RCV1 Collection: si tratta di una collezione di documenti di notizie (ogni notizia contiene titolo, data e corpo). Si ha, nella collezione:

- $N = \# \text{ docs} = 800K$
- $L = \text{avg. } \# \text{ tokens per doc} = 200$
- $M = \# \text{ unique terms} = 400k$
- $\text{avg. } \# \text{ bytes per token (including spaces/punctuation)} = 6$
- $\text{avg. } \# \text{ bytes per token (w.o. spaces/punctuation)} = 4.5$
- $\text{avg. } \# \text{ bytes per term} = 7.5$
- $\text{non-positional postings} = 100.000.000$ (somma delle liste)

NB: token $\neq$ term : con token si intende ogni parola che appare nei documenti, mentre con term si intende ogni parola unica (dopo preprocessing) che appare nei documenti. Il motivo per cui il numero medio di byte per term è maggiore di quello per token è che i termini unici tendono ad essere più lunghi e i token brevi come stopwords, che richiedono meno byte, appaiono molto spesso nei documenti ma vengono esclusi dopo il preprocessing (ad esempio, "the" è un token molto comune ma non è un term unico).

Si noti come da N e L si possa stimare il numero totale di token nei documenti, che è $N * L = 160.000.000$. E allora perché "solo" 100.000.000 di postings? Perché dopo il preprocessing, molti token vengono rimossi (ad esempio, stopwords) o le parole vengono lemmatizzate (ad esempio, "running" diventa "run"), riducendo così il numero totale di token che contribuiscono alle liste di postings.

NB: per tirare fuori queste statistiche è buona pratica lavorare in bash piuttosto che importare subito tutto in pandas. Questo perché con pandas staremmo caricando tutto in RAM, mentre con bash possiamo lavorare streaming riga per riga, evitando di sovraccaricare la memoria. Se volessi **contare il numero totale di token** in tutti i documenti, potrei usare un comando come `cat file.txt | tr ' ' '\n' | wc -l`.

Qui `cat` legge il file, `tr` trasforma gli spazi in nuove linee (quindi ogni token sarà su una nuova riga), e `wc -l` conta il numero di linee, che corrisponde al numero di token (pipe | manda l'output al comando successivo). Quindi per trovare l'avv. # term per document divido il numero totale di token per il numero di documenti. Se volessi invece **contare il numero di unique terms**, potrei usare `cat file.txt | tr ' ' '\n' | sort | uniq | wc -l`. Qui `sort` ordina i token, `uniq` rimuove i duplicati adiacenti (quindi dopo il sort, i termini unici rimangono), e `wc -l` conta il numero di linee, che corrisponde al numero di unique terms. Sort necessario perché `uniq` non elimina tutti i duplicati in generale, ma solo righe uguali consecutive.

External Memory Indexing Algorithms

Ipotizzando 8 byte di spazio per ogni coppia (term, docID), avremmo bisogno di 800 MB solo per memorizzare l'array da ordinare per costruire l'inverted index. Ad oggi la collezione RCV1 è considerata di medie/piccole dimensioni, ma tipicamente si ha a che fare con collezioni di dimensioni molto maggiori, che non possono essere gestite interamente in RAM.

Necessitiamo quindi lavorare anche con il disco.

Sappiamo chiaramente che il disco non può semplicemente sostituire la RAM in quanto:

- è molto più lento
- esiste il problema dei **seek**: prima che un disco legga o scriva dati, deve posizionare la testina sul punto corretto del disco (seek time), e questo è un processo meccanico che richiede tempo. Di conseguenza sarebbe molto inefficiente eseguire l'ordinamento direttamente sul disco, poiché ogni accesso ai dati comporterebbe un seek, rallentando drasticamente il processo. Sempre per via del seek, il disco è **block-based**: leggere e scrivere dati in pochi blocchi grandi è molto più efficiente che leggere e scrivere dati in piccoli blocchi, poiché riduce il numero di seek necessari.

NB: Anche se non c'entra ancora una mazza il Croce ci insiste: è molto più economico comprare e usare più macchine (scalabilità orizzontale) che comprare una macchina più potente (scalabilità verticale), anche a parità di prestazioni. Ciò risulta particolarmente utile anche in termini di Fault Tolerance. (ci si prepara quindi all'indicizzazione su larga scala e MapReduce).

BSBI (Blocked Sort - Based Indexing)

L'idea alla base di BSBI, dato che in RAM non riusciamo ad infilare tutti i 100.000.000 di postings per RCV1 per ordinarli, è quello di **dividere i dati in blocchi più piccoli** che possono essere gestiti in RAM, ordinare ciascun blocco separatamente (sempre in RAM), e poi unire i blocchi ordinati per ottenere l'inverted index finale.

Nell'esempio ipotizziamo che un blocco contenga circa 10k record, quindi un totale di 10 blocchi, ognuno gestibile in RAM. Per ogni blocco:

- definisci i postings (coppie termID, docID) per i documenti che rientrano in quel blocco
- ordina i postings in RAM (ad esempio, usando Quicksort)
- scrivi il blocco ordinato su disco (si sfrutta il fatto che il disco è block-based e quindi scrivere blocchi grandi è più efficiente, meno accesso al disco possibile). Solo alla fine, si effettua il **merge di tutti i blocchi ordinati**.

BSBINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5       $\text{BSBI-INVERT}(block)$ 
6       $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 
```

Con BSBI-invert(block) si intende proprio la costruzione dell'inverted index parziale.

Come fare il merge finale tra i termini nel dizionario? Binary merges: abbiamo a disposizione quindi k blocchi ordinati, dove ogni blocco rappresenta un inverted index parziale (dizionario ordinato + posting list). Per fare il merge dei dizionari ordinati possiamo usare il merge visto in precedenza per ordinare array: dati due blocchi di dimensione n e m, il loro merge costerà $O(n + m)$. Dati k blocchi ordinati, facendo binary merge si creano diversi livelli di merge, in particolare $\log(k)$ livelli. Dato che ad ogni livello si processano tutti i termini, il costo totale del merge sarà $O(T \log(k))$ dove T è il numero totale di termini.

Es.: 8 blocchi B1, B2, ..., B8 ordinati. Primo merge R1 = (B1,B2), R2 = (B3,B4), R3 = (B5,B6), R4 = (B7,B8) --> 4 blocchi ordinati. Secondo merge Q1 = (R1,R2), Q2 = (R3,R4) --> 2 blocchi ordinati. Terzo merge (Q1,Q2) --> 1 blocco ordinato finale. In totale 3 livelli di merge, che corrispondono a $\log_2(8) = 3$.

IMMAGINE

Esiste un metodo che richiede sempre $O(T \log(k))$ per il merge ma è più efficiente in quanto si lavora contemporaneamente su tutti i blocchi senza dover scrivere di volta in volta i risultati dei merge intermedi su disco, si parla di **multi-way merge**. L'idea è di sfruttare una Coda con Priorità:

- si aprono tutti i blocchi contemporaneamente e si cerca il termine più piccolo tra tutti i blocchi (si usa la coda, tempo $O(\log(k))$)
- si uniscono le posting list di quel termine e si passa al termine successivo (tempo $O(\log(k))$ per inserire il nuovo termine nella coda).

In questo modo si evita di leggere e scrivere su disco i risultati intermedi del merge risparmiando moltissimo.

Es.: blocchi ordinati B1 : brutus, caesar, noble; B2 : ambitious, caesar, with; B3 : brutus, julius, killed. All'inizio i termini correnti sono B1 --> brutus, B2 --> ambitious, B3 --> brutus. Questi sono salvati in coda, e il più piccolo è ambitious che si estrae usando l'operazione della coda di ricerca del minimo ($\log(k)$). Si unisce la posting list di ambitious e si passa alla nuova situazione B1 --> brutus, B2 --> caesar, B3 --> brutus (qui l'aggiunta di caesar alla coda richiede $\log(k)$). Il più piccolo è brutus, che compare in B1 e B3 --> si prendono entrambe le posting list e si fondono e aggiungono al documento finale e così via.

Come fare invece il merge finale tra le posting list? Dal momento che i blocchi sono contigui e creati a partire dai documenti già ordinati per DocID, fare il merge tra blocchi in termini di posting list richiede solo di concatenarle. Es.: brutus compare in B1 e B3, le posting list sono rispettivamente [1,2] e [5,7], il merge è semplicemente [1,2,5,7], e non poteva accadere che B1 avesse [1,5] e B3 avesse [2,7] perché i blocchi sono stati creati a partire da documenti ordinati per DocID.

Costi: Ipotizzando $N = 100.000.000$ di postings, $B = 10$ blocchi, si ha $O(N/B \log(N/B))$ per ordinare ogni blocco. Eseguiamo B volte questa operazione, quindi $O(N \log(N/B))$ per ordinare tutti i blocchi. Il merge finale è costante per il merge tra posting lists, ma richiede $O(T \log(k))$ per il merge dei dizionari, dove T è il numero totale di termini e k è il numero di blocchi.

NB: Il prof a lezione insiste che usando alberi binari per il dizionario il costo per merge è in realtà logaritmico nel numero di termini (altezza dell'albero del dizionario) ma non torna, merge tra AVL è lineare...

SPIMI (Single-Pass In-Memory Indexing)

BSBI ha un problema: presuppone che in memoria RAM sia comunque mantenuto un dizionario globale dei vari termID. Questo perché BSBI mappa ogni termine ad un ID numerico,

e ovviamente per fare il merge dei blocchi è necessario sapere che se il termine "brutus" compare in blocchi diversi questo deve avere lo stesso ID, dato che è lo stesso termine. Quindi in pratica in RAM deve essere presente la mappatura globale tra termini e termID. (non l'ho fatto vedere negli esempi ma di base i vari termini sono mappati in degli id univoci, dei valori interi).

Tuttavia questo dizionario globale cresce dinamicamente e può diventare molto grande, diventando collo di bottiglia.

SPIMI risolve questo problema. Vi sono tre idee chiave dietro l'algoritmo che lo rendono migliore di BSBI:

- Genera blocchi in base a quanto spazio in RAM è disponibile, quindi non è necessario stimare a priori la dimensione dei blocchi (come invece in BSBI).
- Per ogni blocco, si costruisce un dizionario locale per cui non mappiamo le stringhe a ID ma le lasciamo così come sono --> si evita la necessità di un dizionario globale.
- Non si accumulano per ogni blocco le coppie (termID, docID) ma tutte insieme per poi effettuare un sorting alla fine, ma si costruisce direttamente l'inverted index allo scorrere dei token nel blocco.

```
SPIMI-INVERT(token_stream)
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTODICTIONARY(dictionary, term(token))
7          else postings_list = GETPOSTINGSLIST(dictionary, term(token))
8          if full(postings_list)
9              then postings_list = DOUBLEPOSTINGSLIST(dictionary, term(token))
10     ADDTOPOSTINGSLIST(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

■ Merging of blocks is analogous to BSBI.

Descrizione dello pseudocodice: si parte con un nuovo file e una hash table vuota per blocco. Fintanto che c'è memoria disponibile in RAM, si scorrono i token. Se il term(token) (radice secondo lemmatizzazione etc...) non è già nel dizionario, significa che non ha posting list definita --> *posting_list* = AddToDictionary(...). Se il termine invece è già presente, allora si aggiunge semplicemente il docID alla posting list del termine (**qui il cuore di SPIMI, non salvi**

tuple sparse da ordinare dopo, ma già inizia a costruire la posting list per ogni termine). Quando la posting list è piena, si espande via DoublePostingList(...) (L'idea è che le posting list crescono dinamicamente). Infine, quando la memoria è piena, ordini i termini del dictionary (NB: si ordinano solo i term del dictionary, non tutte le coppie come in BSBI, molto più veloce) e scrivi su disco il blocco.

Riguardo il merging tra blocchi alla fine, è analogo a BSBI.

SPIMI migliora ancora se comprimi termini e posting lists, lo vedremo successivamente.

Distributed Indexing

Per collezioni enormi (tipo web-scale), non basta più una singola macchina: serve un cluster di macchine.

Importante puntualizzare di nuovo come anche motori come Google, Bing, etc... usano tantissime macchine normali, non necessariamente tutti supercomputer (invece di una macchina "perfetta", conviene usare moltissime macchine commodity e progettarsi il sistema in modo da tollerare i guasti).

Perché i guasti sono inevitabili? Perché più macchine si usano, più è probabile che una di esse fallisca. Immaginiamo un sistema con 1000 nodi, dove ognuno ha 99.9% di uptime.

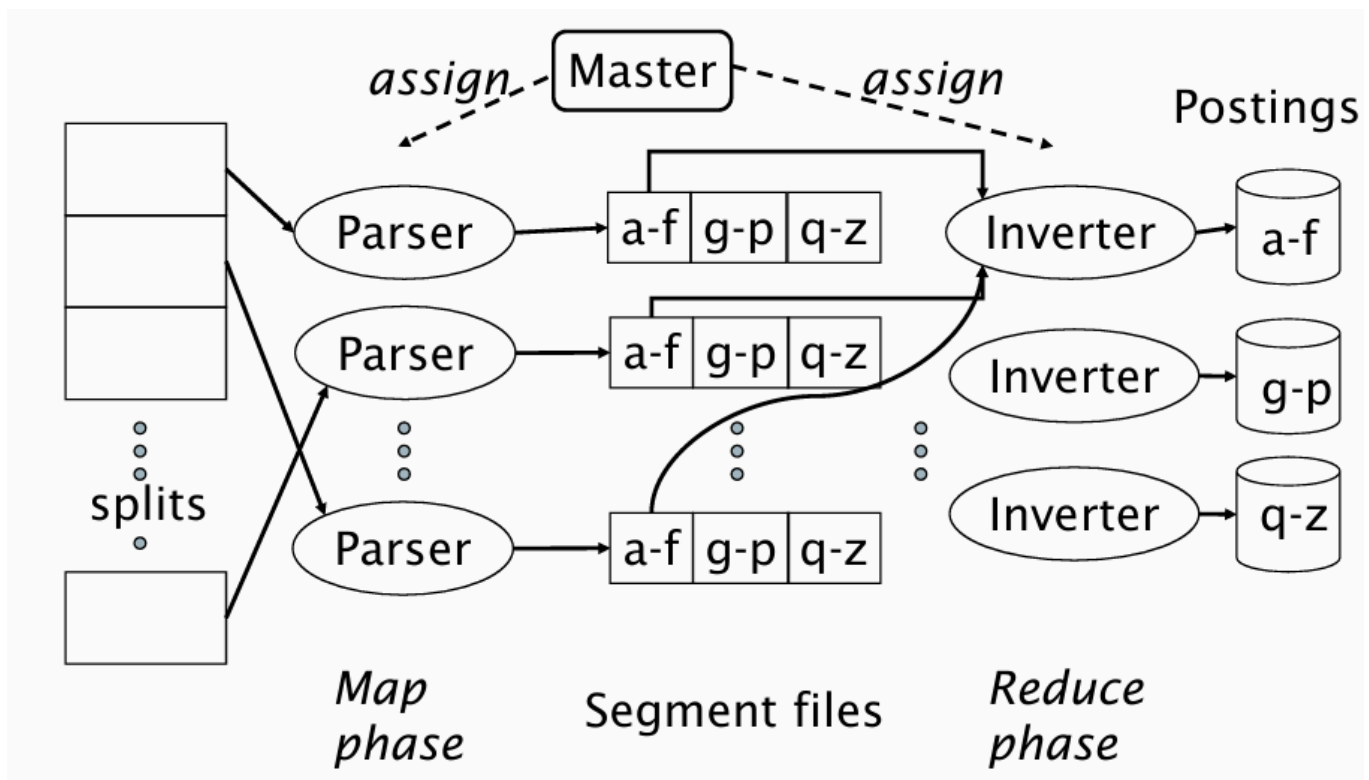
La probabilità che un nodo fallisca in un dato momento è 0.001 (1-0.999). La probabilità che almeno un nodo fallisca è $1 - (\text{probabilità che tutti i nodi funzionino}) = 1 - (0.999^{1000}) \approx 0.63$, quindi c'è circa il 63% di probabilità che almeno un nodo fallisca in un dato momento.

Quindi in sistemi molto grandi il problema non è se una macchina fallirà, ma quale e quando!

Come sfruttare quindi tante macchine insieme, sapendo che qualcuna fallirà prima o poi whp?

Idea: mantenere una macchina centrale (**master**) che assegna e coordina il lavoro alle altre macchine (**slaves**). Il master è considerato "sicuro" (non fallisce). Il lavoro assegnato dal master è suddiviso in **task paralleli**, ognuno assegnato ad uno slave libero. Se uno slave fallisce, il master rileva il fallimento e riassegna il task ad un altro slave. Ognuno dei task può essere svolto in parallelo.

Esistono due principali tipi di slaves, legati ai due tipi di task su cui possono lavorare: **Parsers** (per la fase di **Map**) e **Inverters** (per la fase di **Reduce**). I parsers si occupano di leggere i documenti, fare il preprocessing e generare coppie (term, docID), mentre gli inverters si occupano di prendere queste coppie e costruire l'inverted index. Vediamo più precisamente come funziona.



Data una collezione (enorme) di documenti, il master li suddivide in vari splits (l'equivalente dei blocchi di BSBI) e assegna ognuno di essi a un parser. Ogni parser legge i documenti del suo split, fa il preprocessing e genera coppie (term, docID). **Molto importante:** i parser generano j partizioni. Date le coppie di (term, docID), ogni parser li divide per range di termini, nell'esempio $j = 3$ partizioni a-f | g-p | q-z, che vengono salvati su disco come documenti separati. A questo punto entrano in gioco gli inverter: ogni inverter prende una partizione (ad esempio: a-f), li ordina e costruisce l'inverted index per quella partizione. Alla fine si ottiene quindi un indice diviso per range di termini, che può essere facilmente unito per ottenere l'inverted index finale.

Molto importante puntualizzare che ogni parser e ogni inverter lavorano parallelamente in quanto indipendenti tra loro --> parallelismo massimo. Importante inoltre dire che se il master si rompe allora tutto il sistema si rompe, ma se si rompono i parser o gli inverter, il master se ne accorge e riassegna il lavoro ad altri nodi, quindi il sistema è tollerante ai guasti. Inoltre i file finali dovrebbero avere un sistema di backup.

L'inverted index così costruito non è necessariamente nella forma migliore per poter fare le query. Distinguiamo in questo senso due tipi di indice:

- **Term.Partitioned Index:** ogni partizione contiene un range di termini, e per ogni termine è presente la sua posting list completa. Questo è esattamente il risultato che otteniamo secondo la strategia vista prima, per cui ogni macchina gestisce un gruppo di termini e contiene tutte le loro posting lists.
- **Document-Partitioned Index:** ogni partizione (macchina) gestisce solo un gruppo di documenti. **NB:** nel doc-partitioned index non cambia la struttura dell'indice, ma solo dove è memorizzato: ogni macchina ha un inverted index parziale. Se ad esempio nel term-

partitioned la macchina 1 gestisce i termini a-f, la macchina 2 g-p, nel document partitioned la macchina 1 gestisce i documenti 1-100k, la macchina 2 i documenti 100k-200k, ognuna con il proprio inverted index parziale.

Nelle applicazioni reali, molti motori di ricerca usano Document-Partitioned Index. Alcuni dei vantaggi infatti sono:

- **Load Balancing:** con un Term-Partitioned Index, alcuni termini (ad esempio, stopwords) possono essere estremamente frequenti e quindi creare posting list enormi che sovraccaricano la macchina che le gestisce. Con un Document-Partitioned Index, invece, ogni macchina gestisce un gruppo di documenti, quindi il carico è più bilanciato.
- **Fault Tolerance:** con un Term-Partitioned Index, se una macchina che gestisce un termine molto frequente fallisce, si perde l'accesso a quel termine e quindi a tutti i documenti che lo contengono. Con un Document-Partitioned Index, se una macchina fallisce, si perde solo l'accesso a un gruppo di documenti, ma non a termini specifici, quindi il sistema è più tollerante ai guasti.
- **Fondamentale nel caso in cui si abbia un indice dinamico, che viene aggiornato di frequente con nuovi documenti (ad esempio, Twitter):** dato che ogni macchina gestisce un sottoinsieme di documenti, si potrebbero di volta in volta assegnare i documenti più recenti a macchine specifiche, non dovendo aggiornare tutto l'indice in tutte le macchine (come invece accadrebbe con un Term-Partitioned Index, dove ogni nuovo documento potrebbe contenere termini che vanno ad aggiornare le posting list di tutti i blocchi).

MapReduce

L'algoritmo visto in precedenza è un esempio di MapReduce. MapReduce è un framework per fare calcoli distribuiti senza dover gestire comunicazione tra macchine, fault tolerance e parallelismo. Con MapReduce, l'utente scrive solo le due funzioni chiave, **Map** e **Reduce**, e il framework si occupa di tutto il resto. In generale:

- **Map** prende un input e produce una lista di coppie chiave-valore (input $\rightarrow$ list(k,v)). Nel nostro caso, la funzione Map prende in input un documento e produce una lista di coppie (term, docID).
- **Reduce** prende una lista di valori associati a una chiave e produce un output ($(k, list(v)) \rightarrow output$). Nel nostro caso, la funzione Reduce prende in input una lista di docID per un termine e produce in output l'inverted list per quel termine.

Dynamic Indexing

Abbiamo finora assunto che la collezione fosse statica, ma in realtà i documenti vengono aggiunti, modificati e cancellati continuamente. Come gestire quindi un indice dinamico, dove aggiornare dizionario e posting lists? (Es.: se arriva un nuovo doc con il termine "brutus" dovrei

aggiornare la posting list di "brutus" oppure crearne una nuova se non esiste in quella macchina).

Idea: virtualmente vediamo l'indice (distribuito sempre in varie macchine) come suddiviso in due sottoinsiemi: un **Main Index** grande, su disco, che contiene la maggior parte dei documenti e un **Auxiliary Index** più piccolo, in RAM, che contiene i documenti più recenti.

- **Insert:** Quando arriva un nuovo documento, lo si aggiunge all'Auxiliary Index senza mai toccare il Main Index. Il vantaggio in questo senso è che l'inserimento sarà molto veloce, in quanto non devo aggiornare il Main Index. Quando l'utente effettua una query, si interroga sia il Main Index che l'Auxiliary Index, e si uniscono i risultati.
- **Delete:** Cancellare subito fisicamente un documento sarebbe troppo costoso, per cui si associa semplicemente un flag di invalidazione ad ogni documento. Quando si effettua una query, i documenti con il flag ad 1 vengono filtrati ed esclusi dai risultati.

Al crescere dell'Auxiliary Index, sarà necessario fare un merge con il Main Index, unendo i documenti più recenti con quelli più vecchi.

Questa idea è purtroppo impraticabile, dal momento che fare merge tra Main e Aux richiede di mettere mano su tantissimi dati ed è quindi molto costoso. Inoltre Performance scarse durante merge (mentre il motore di ricerca fa merge, il sistema è molto lento e non riesce a rispondere alle query, I/O pesante...). Si noti come una soluzione teorica al problema sarebbe quella di mantenere un file per ogni posting list, per cui fare merge richiederebbe un semplice append (similmente a quanto visto con BSBI), in quanto i docID nell'auxiliary sono sempre successivi a quelli del main. Tuttavia mantenere un file per ogni posting list è impraticabile per il sistema operativo.

Assumiamo quindi che l'indice sia salvato in un grosso file (anche se nella realtà le posting lists sono separate tipicamente in base alla lunghezza, quindi tipo tutte quelle di lunghezza 1 in un file etc...).

Logarithmic Merge

Quindi il metodo Main + Aux è semplice, ma non scala. Un'idea più scalabile è quella del **Logarithmic Merge**, per cui si mantengono più indici di dimensioni crescenti. Invece di avere solo Main + Aux, si hanno più livelli di indici: Z0 (piccolo), I0 (medio), I1 (più grande), I2 (ancora più grande) e così via. Z0 è l'indice più piccolo, in RAM, nel quale vengono inseriti i documenti più recenti. I vari I_j invece si trovano nel disco. Quando Z0 raggiunge una certa dimensione, si hanno due casi:

- Se I0 non esiste --> Z0 diventa I0 e viene spostato in disco
- Se I0 esiste --> si fa merge tra Z0 e I0, che diventano un nuovo indice I1, più grande di I0. Di nuovo: se I1 non esiste, allora I1 diventa il nuovo indice, altrimenti si fa merge tra I1 e il

nuovo indice per ottenere I_2 , e così via.

```
LMERGEADDTOKEN(indexes,  $Z_0$ , token)
1   $Z_0 \leftarrow \text{MERGE}(Z_0, \{token\})$ 
2  if  $|Z_0| = n$ 
3      then for  $i \leftarrow 0$  to  $\infty$ 
4          do if  $I_i \in \text{indexes}$ 
5              then  $Z_{i+1} \leftarrow \text{MERGE}(I_i, Z_i)$ 
6                  ( $Z_{i+1}$  is a temporary index on disk.)
7                   $\text{indexes} \leftarrow \text{indexes} - \{I_i\}$ 
8              else  $I_i \leftarrow Z_i$  ( $Z_i$  becomes the permanent index  $I_i$ .)
9                   $\text{indexes} \leftarrow \text{indexes} \cup \{I_i\}$ 
10                 BREAK
11          $Z_0 \leftarrow \emptyset$ 

LOGARITHMICMERGE()
1   $Z_0 \leftarrow \emptyset$  ( $Z_0$  is the in-memory index.)
2   $\text{indexes} \leftarrow \emptyset$ 
3  while true
4  do LMERGEADDTOKEN(indexes,  $Z_0$ , GETNEXTTOKEN())
```

Nello pseudocodice: se arriva un nuovo token, si aggiunge a Z_0 . Se Z_0 è pieno (dimensione n) allora per ogni livello di indice a partire da 0 si fa il merge e si crea quindi il nuovo indice da mettere su disco. Fatto ciò, si svuota Z_0 e si continua ad aggiungere nuovi documenti.

Vediamo quanto ci costa. Sia T il numero totale di postings (dimensione totale dei dati) e n dimensione dell'auxiliary index (sempre in termini di posting). Anzitutto, è fondamentale capire che ogni posting delle posting list viene riscritta solo durante un merge (quando si fa il merge, si riscrivono infatti le posting list coinvolte).

Nel caso banale senza logarithmic merge, avevamo un main ed un aux. In questo caso, ipotizzando come detto di avere T postings totali e Aux di dimensione n , il numero totale di merge per arrivare a un main con T postings è T/n (es. T 1000 postings e n 10, aux si riempie quindi primo merge metto 10 in main, poi si riempie ancora aux con altri 10, secondo merge metto 20 in main e così via per un totale di $1000/10 = 100$ merge). Ma ogni merge costa $O(T)$ in quanto si riscrivono tutte le posting list, quindi il costo totale è $O(T^2/n)$ (non scalabile).

Nel caso del logarithmic merge invece, una posting list viene riscritta solo quando il suo blocco è coinvolto in un merge, ossia "sale di livello". Quindi al più si ha un merge per livello, ed il

numero di livelli è $\log(T/n)$ ($n \rightarrow 2n \rightarrow 4n \rightarrow 8n \rightarrow \dots \rightarrow T \Rightarrow 1 \rightarrow 1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow \dots \rightarrow T/n$).
Ogni merge costa sempre al più $O(T)$ -> **costo totale logarithmic merge** $O(T \log(T/n))$.

Tuttavia nel logarithmic merge esiste un **tradeoff** intrinseco: mentre con la versione main + aux in caso di query si interrogano solo due indici, con il logarithmic è necessario interrogare tutti gli indici ($Z_0, I_0, I_1, I_2, \dots$), al più $\log(T/n)$ indici.

Un ulteriore problema nell'uso di indici multipli: **è complesso fare statistiche sui termini delle collezioni**. Infatti, quando si ha un main index e un auxiliary index (o ancora più livelli) allora i dati non sono più virtualmente in un unico indice.

Questo è un problema per i motori di ricerca, che fanno largo uso di statistiche globali del tipo "quante volte appare un termine nell'intera collezione", di modo da fare ranking (ordine dei risultati, capire quali documenti sono più rilevanti per una query, usa TF-IDF) o spell correction. Spell correction infatti funziona come segue: se cerco ad esempio "recieve" invece di receive, il motore di ricerca cerca termini simili a "recieve" (ad esempio, usando edit distance) e li ordina in base alla frequenza globale del termine nella collezione (quindi se "receive" appare 1000 volte e "recieve" 10 volte, allora "receive" è più probabile che sia il termine corretto e lo suggerisce).

Spesso per aggirare il problema, si considera solo il main index per fare le statistiche (dal momento che questo è il più grande e stabile), tuttavia si perde una statistica accurata. Come fanno allora davvero i motori di ricerca?

1. **Aggiornamenti incrementali**: (quindi o main + aux o logarithmic merge).
2. **Rebuild periodico dell'indice**: ogni tot. tempo si ricostruisce l'intero indice da zero, partendo da tutti i documenti (quindi anche quelli più recenti). In questo modo si ottiene un indice unico e aggiornato, con statistiche accurate, si pulisce la frammentazione e gli errori accumulati. Fondamentale sapere che il rebuild è un processo molto costoso che per questo **avviene in background** (quindi non impatta sulle query), quando è pronto si switcha semplicemente all'altro indice, che viene eliminato -> ZERO DOWNTIME.

Esempio reale: EarlyBird (Twitter)

EarlyBird è il motore di ricerca di Twitter, che indicizza i tweet in tempo reale. Su Twitter milioni di tweet vengono pubblicati ogni giorno, e gli utenti si aspettano di poterli cercare immediatamente. Inoltre sono i tweet più recenti ad essere più importanti.

Requisiti:

- Low latency: i risultati devono essere restituiti in pochi millisecondi
- high ingestion rate: tanti nuovi tweet ogni secondo

- concurrent reads and writes: molti utenti che fanno query e tanti nuovi tweet che arrivano contemporaneamente
- dominance of temporal signal: i tweet più recenti sono più rilevanti

IDEA: non usare un unico indice grande e merge pesanti, ma **mantenere più indici piccoli (segmenti)**. Di volta in volta solo un segmento è attivo in scrittura, pronto in RAM, mentre gli altri sono in lettura su disco. Ogni volta che arriva un nuovo tweet, questo viene aggiunto alla posting list via append del segmento attivo. Quando il segmento si riempie, viene salvato in memoria secondaria senza fare immediatamente merge (i merge sono gestiti in modo diverso dal logarithmic merge, non viene spiegato come). Per quanto riguarda il requisito temporale, semplicemente alla richiesta **le posting lists sono attraversate al contrario** (in quanto i nuovi tweet, che sono i più rilevanti vengono appesi alla fine della posting list).

Positional e n-grams

Con il positional index non si hanno solo più coppie (term, docID), ma anche le posizioni dei termini nel documento (term, docID, [pos1, pos2]) per un inverted index del tipo term -->[docID -->[posizioni]] (ogni posting contiene docID e la lista di posizioni). Qui quindi il problema visto in precedenza si ripresenta, ma ancora più grande... moltissimi più elementi da ordinare!

Lo stesso vale se volessimo creare indici per n-grams (utile ad esempio per la spell correction, cat --> ca, at; dog --> do, og), tantissimi più indici da ordinare e tante associazioni (es. ca punta a tante parole diverse e quindi a tante posting list diverse).